



Java

Clase 07 | Herencia y polimorfismo

Índice

Clase 07

Clase 7 | Herencia y polimorfismo

- Concepto de herencia en Java.
- Creación de subclases y superclases.
- Uso de super y this.
- Polimorfismo y sobrescritura de métodos.
- Clases abstractas e interfaces.
- Ejemplos prácticos aplicados al proyecto.

Clase 08

Clase 8 | Manejo de Excepciones y Módulos

- Manejo de excepciones en Java.
- Uso de try, catch, finally.
- Creación de excepciones personalizadas.
- Organización del código con paquetes y módulos.
- Importación y uso de clases externas.

Objetivos de la Clase



Comprender la herencia

Entender qué es una superclase, una subclase y cómo se relacionan.



Crear subclases

Aprender a extender una superclase usando extends.



Explorar el polimorfismo

Tratar diferentes subclases a través de una referencia de la superclase.

Herencia



Concepto de Herencia en Java

La herencia es uno de los pilares fundamentales de la programación orientada a objetos en Java. Es un mecanismo que permite que una clase (llamada subclase o clase hija) pueda heredar características y comportamientos de otra clase (llamada superclase o clase padre).

Superclase y Subclase

Superclase.

Es aquella que contiene los atributos y métodos comunes que serán heredados por otras clases. Por ejemplo, una clase "Vehículo" podría ser una superclase que define características básicas como "color", "marca" y "modelo".

Subclases.

Son aquellas que heredan todas estas características de la superclase y pueden agregar sus propios atributos y métodos específicos. Siguiendo el ejemplo anterior, "Automóvil" y "Motocicleta" podrían ser subclases de "Vehículo", agregando características específicas como "número de puertas" o "tipo de manubrio".

Beneficios de la Herencia

1 Reutilización de Código

Evita duplicar código al reutilizar las características y comportamientos de una superclase.

3 Extensibilidad

Facilita agregar nuevas características o funcionalidades a las subclases.

2 Organización

Estructura el código de manera lógica, formando una jerarquía de clases.

4 Mantenimiento

Simplifica la modificación y el mantenimiento del código al centralizar cambios en la superclase.

Creación de Subclases y Superclases

Aquí "Producto" es la superclase de "Bebida" y "Comida":

1 Declarar herencia

Usar la palabra clave extends para que una clase herede de otra.

```
class Producto {  
    String nombre;  
    double precio;  
}  
class Bebida extends Producto {  
    int volumenML;  
}  
class Comida extends Producto {  
    int pesoGramos;  
}
```


Creando objetos: Comida, Producto y Bebida

1 Comida

Al crear un objeto de tipo Comida, estás instanciando una subclase específica que hereda las propiedades de Producto. Este objeto podrá manejar atributos específicos de alimentos como el peso en gramos:

```
Comida miComida = new  
Comida();
```

2 Producto

La instanciación de un objeto Producto representa la clase base o superclase. Este objeto contendrá las propiedades comunes como nombre y precio que heredarán tanto Comida como Bebida:

```
Producto miProducto =  
new Producto();
```

3 Bebida

Al instanciar un objeto de tipo Bebida, creas una subclase que hereda de Producto pero maneja atributos específicos para líquidos como el volumen en mililitros:

```
Bebida miBebida = new  
Bebida();
```

Uso de super y this

La palabra clave **super** permite acceder a miembros (atributos y métodos) de la clase padre, mientras que **this** hace referencia al objeto actual de la clase donde se está usando.



Uso de this

this

Hace referencia al objeto actual. Se usa para acceder a atributos y métodos de la clase actual.

```
class Producto {  
    String nombre;  
  
    Producto(String nombre) {  
        this.nombre = nombre;    // 'this' referencia al atributo de la clase  
    }  
  
    void mostrarInfo() {  
        this.imprimirNombre();    // 'this' llama a método de la clase  
    }  
}
```

Uso de super:

super

Hace referencia a la superclase. Se usa para llamar al constructor o métodos de la superclase.

```
class Bebida extends Producto {
    int volumenML;
    Bebida(String nombre, int volumenML) {
        super(nombre);          // Llama al constructor de Producto
        this.volumenML = volumenML;
    }
    @Override
    void mostrarInfo() {
        super.mostrarInfo();    // Llama al método de Producto
        System.out.println("Volumen: " + volumenML);
    }
}
```

Polimorfismo

Polimorfismo en Java

El polimorfismo es uno de los pilares fundamentales de la programación orientada a objetos en Java. En esencia, permite que un objeto pueda comportarse de diferentes formas según el contexto, proporcionando flexibilidad y versatilidad al código.



Polimorfismo en Java

Características principales:

- Permite tratar objetos de diferentes clases a través de una interfaz común
- Facilita la creación de código más genérico y reutilizable
- Soporta el principio de sustitución: una clase hija puede ser tratada como su clase padre

Ventajas:

- Mejora la mantenibilidad del código al reducir la duplicación
- Aumenta la flexibilidad en el diseño de aplicaciones
- Facilita la extensión del código sin modificar el existente
- Permite escribir métodos que pueden trabajar con objetos de diferentes clases de manera transparente



La Magia del Polimorfismo

El polimorfismo permite que un objeto se comporte de diferentes maneras dependiendo de su tipo real. Aunque todos los objetos en el array son del tipo **Producto**, cada uno se comporta como su subclase específica: **Comida** o **Bebida**.

En el código, se utiliza el operador **instanceof** para determinar el tipo de cada objeto y luego se realiza una conversión de tipo. Esto permite acceder a los métodos específicos de cada subclase y obtener la información relevante.

Comportamiento Polimórfico de un Producto

Ejemplo de Array de Productos

Considera un array de objetos **Producto** que contiene tanto **Comida** como **Bebida**.

```
Producto[] productos = {  
    new Comida("Pizza", 10, 20240510),  
    new Bebida("Jugo de naranja", 5, 1000),  
    new Comida("Hamburguesa", 8, 20240515)  
};
```

Comportamiento Polimórfico de un Producto

Iteración y Comportamiento

Al iterar sobre el array, podemos usar polimorfismo para que cada objeto se comporte según su tipo específico.

```
for (Producto producto : productos) {  
    if (producto instanceof Comida) {  
        Comida comida = (Comida) producto;  
        System.out.println("Comida: " + comida.getNombre() + ", Fecha de vencimiento: " +  
comida.getVencimiento());  
    } else if (producto instanceof Bebida) {  
        Bebida bebida = (Bebida) producto;  
        System.out.println("Bebida: " + bebida.getNombre() + ", Volumen: " +  
bebida.getVolumenML());  
    }  
}
```

Clases abstractas

Clases Abstractas en Java

Una clase abstracta es una clase que no puede ser instanciada directamente con 'new'. Actúa como una plantilla para otras clases, definiendo una estructura común que las subclasses deben seguir.

Características

Puede contener tanto métodos abstractos (sin implementación, que las subclasses deben implementar) como métodos concretos (con implementación completa). Se declara usando la palabra clave 'abstract' y puede tener constructores y atributos.

Uso

Se utiliza cuando queremos definir un comportamiento común para un grupo de clases relacionadas. Por ejemplo, en un sistema de figuras geométricas, podríamos tener una clase abstracta 'Figura' con un método abstracto 'calcularArea()'.
`calcularArea()`



¿Por qué Clases Abstractas?

Las clases abstractas son como plantillas o moldes que definen una estructura común, pero no pueden existir por sí mismas. Imaginemos un zoológico: no podemos tener un "animal" genérico, sino que cada animal debe ser de una especie específica como un tigre, un elefante o un pingüino. La clase abstracta "Animal" define características comunes (como comer o dormir), pero necesitamos clases concretas para crear animales reales.

Ejemplo de Clase Abstracta

Clase Abstracta Animal

Define las características comunes a todos los animales.

```
abstract class Animal {  
    protected String nombre;  
    protected String especie;  
  
    public abstract void comer();  
    public abstract void dormir();  
    public String getNombre() {  
        return nombre;  
    }  
    public String getEspecie() {  
        return especie;  
    }  
}
```

Ejemplo de Clase Abstracta

Clase Tigre: Hereda de la clase Animal e implementa los métodos abstractos.

```
class Tigre extends Animal {  
    public Tigre(String nombre) {  
        this.nombre = nombre;  
        this.especie = "Tigre";  
    }  
  
    @Override  
    public void comer() {  
        System.out.println("El tigre está comiendo carne.");  
    }  
  
    @Override  
    public void dormir() {  
        System.out.println("El tigre está durmiendo.");  
    }  
}
```


Ejemplo práctico: Clases Abstractas

La clase abstracta **Figura** define un método abstracto **calcularArea()**, obligando a las subclases a implementar su lógica específica.

La clase **Circulo** hereda de **Figura** y proporciona una implementación concreta para **calcularArea()**, calculando el área de un círculo.

```
abstract class Figura {
    protected String nombre;
    // Método abstracto - debe ser implementado por las
    subclases
    public abstract double calcularArea();
    // Método concreto - implementación común para todas las
    subclases
    public String getNombre() {
        return nombre;
    }
}

class Circulo extends Figura {
    private double radio;
    @Override
    public double calcularArea() {
        return Math.PI * radio * radio;
    }
}
```


Problema: Herencia y Polimorfismo en el Catálogo

⚠ Necesitamos implementar un sistema de productos donde cada tipo específico (Bebida, Comida) herede de una clase base Producto, permitiendo manejar diferentes productos de manera polimórfica en el catálogo.

```
// Clase base
abstract class Producto {
    protected String nombre;
    protected double precio;

    public abstract void
mostrarDetalles();
}
```

```
// Clase derivada Bebida
class Bebida extends Producto {
    private boolean caliente;

    @Override
    public void mostrarDetalles() {
        System.out.println("Bebida: " + nombre +
            " - Precio: $" + precio +
            " - Caliente: " + caliente);
    }
}
```

Problema: Herencia y Polimorfismo en el Catálogo

```
// Clase derivada Comida
class Comida extends Producto {
    private boolean paraLlevar;

    @Override
    public void mostrarDetalles() {
        System.out.println("Comida: " +
            nombre +
            " - Precio: $" +
            precio +
            " - Para llevar: " + paraLlevar);
    }
}
```

```
// Uso polimórfico
ArrayList catalogo = new ArrayList<>();
catalogo.add(new Bebida("Café", 2.5,
    true));
catalogo.add(new Comida("Croissant", 3.0,
    true));

// Mostrar todos los productos de forma
polimórfica
for(Producto p : catalogo) {
    p.mostrarDetalles();
}
```

Interfaces

Interfaces en Java

Un interface actúa como un contrato formal que especifica qué métodos debe implementar una clase. Es similar a una clase abstracta, pero a diferencia de esta, una interface solo puede contener métodos abstractos (sin implementación) y constantes. Cuando una clase implementa una interface, está obligada a proporcionar código para todos los métodos declarados en ella.

Características

A diferencia de las clases normales, las interfaces solo pueden contener declaraciones de métodos (sin implementación) y constantes. Todos los métodos son implícitamente públicos y abstractos, y todas las variables son implícitamente public, static y final. Esto asegura que las interfaces sean verdaderos contratos puros sin detalles de implementación.

Solución a la Herencia Múltiple

Java no permite que una clase herede de múltiples clases (herencia múltiple) para evitar problemas de ambigüedad. Sin embargo, las interfaces resuelven esta limitación ya que una clase puede implementar múltiples interfaces. Por ejemplo, una clase Murciélago puede implementar tanto la interface Volador como Mamífero, logrando así una forma segura de herencia múltiple sin los problemas tradicionales de conflictos entre métodos.

Ejemplo de Interfaz en Java

Interface

```
public interface Volador {  
    void volar();  
}
```

Clase

```
public class Pájaro implements Volador {  
    @Override  
    public void volar() {  
        System.out.println("El pájaro está  
volando.");  
    }  
}
```

Problema: Interfaces y Herencia en el Catálogo

⚠ Necesitamos implementar un sistema de productos donde la clase Cafe no solo herede de Bebida, sino que también implemente una interface Preparable, permitiendo definir cómo se prepara cada bebida caliente.

```
// Interface
interface Preparable {
    void preparar();
}

// Clase base
abstract class Producto {
    protected String nombre;
    protected double precio;

    public abstract void mostrarDetalles();
}
```

```
// Clase Bebida
class Bebida extends Producto {
    protected boolean caliente;

    @Override
    public void mostrarDetalles() {
        System.out.println("Bebida: " + nombre +
            " - Precio: $" + precio +
            " - Caliente: " + caliente);
    }
}
```


Problema: Interfaces y Herencia en el Catálogo

```
// Clase Cafe que hereda de Bebida e implementa Preparable
class Cafe extends Bebida implements Preparable {
    private String tipoGrano;
    @Override
    public void preparar() {
        System.out.println("Moliendo granos de " +
tipoGrano);
        System.out.println("Calentando agua a 92°C");
        System.out.println("Preparando café...");
    }
    @Override
    public void mostrarDetalles() {
        System.out.println("Café: " + nombre +
            " - Precio: $" + precio +
            " - Tipo: " + tipoGrano);
    }
}
```

```
// Uso del polimorfismo e
interfaces
Cafe cafe = new Cafe("Espresso",
2.5, "Arábica");
cafe.mostrarDetalles();
cafe.preparar();
```

Material y Recursos Adicionales



Documentación Oracle

Herencia y Polimorfismo en Java.



Videos YouTube

Tutoriales sobre herencia, polimorfismo, clases abstractas e interfaces.



Libros

Lecturas recomendadas sobre POO avanzada en Java.



docs.oracle.com



Inheritance (The Java™ Tutorials > L...

This beginner Java tutorial describes fundamentals of programming in the Java programming language



Preguntas para Reflexionar



Herencia y código

¿Cómo evita la herencia la duplicación de código?



Abstractas vs Interfaces

¿En qué casos usarías una clase abstracta versus una interfaz?



Polimorfismo y colecciones

¿Cómo simplifica el polimorfismo el manejo de colecciones de objetos relacionados?

Resumen: Herencia en Java

Herencia

Base de la jerarquía de clases

Superclase

Define estructura común

Subclase

Especializa comportamiento

Reutilización

Evita duplicación de código

Resumen: Polimorfismo en Java

Definición

Tratar objetos de subclases como instancias de su superclase.

Ventajas

Flexibilidad y extensibilidad en el diseño de sistemas.

Aplicación

Permite manejar objetos relacionados de manera uniforme.

Conclusión



Herramientas poderosas

Herencia y polimorfismo son fundamentales en POO. (y siempre se pregunta sobre esto en entrevistas laborales)



Diseño robusto

Permiten crear sistemas más flexibles y mantenibles.



Práctica continua

Seguir aplicando estos conceptos es clave, ya que son difíciles de entender al comienzo, pero luego no puedes dejar de pensar en ellos al crear un sistema.

Ejercicios prácticos

Situación inicial en TechLab.



Silvia, la Product Owner, observa que actualmente hay una sola clase Producto, pero el catálogo ha crecido y cada tipo de producto tiene características y reglas distintas. Por ejemplo, Té Y Café. Alguna requiere cálculos de descuento particulares, otras tienen restricciones de stock o fecha de vencimiento.

Matías y Sabrina se dan cuenta de que no pueden seguir agregando if y switch dentro de Producto para cada caso especial. Necesitan una forma más elegante de modelar las diferencias: la **herencia** les permitirá crear una clase base Producto y luego Te y Café como subclases que especialicen el comportamiento. El polimorfismo les permitirá tratar a todos ellos como Producto al mismo tiempo que cada uno sabe hacer su trabajo de forma independiente.

Además, en algunos casos se requerirá definir un comportamiento sin implementar completamente la lógica en la superclase, o establecer un contrato de métodos que las subclases deben cumplir. Aquí entran en juego las clases abstractas y las interfaces.

Para cumplir con necesidades el equipo de desarrollo (Matías, Sabrina) y vos deberán:

Ejercicio práctico

Optativo | No entregable

1- Herencia y polimorfismo

Herencia básica:

- Creá la clase abstracta `Producto` con un atributo `nombre` y un método abstracto `calcularPrecioFinal()`.
- Creá `Te` y `Café` que extiendan `Producto`.
- Implementá `calcularPrecioFinal()` en cada subclase.

Polimorfismo:

- Creá un `ArrayList<Producto>` y agregá instancias de `Te` y `Café`.

Iterá sobre la lista y llama a `calcularPrecioFinal()` en cada uno. Observa cómo se ejecuta la versión correspondiente a cada subclase.

Ejercicio práctico

Optativo | No entregable

2- Override e interfaces

Uso de super y override:

- En Cafe, agregá un constructor que llame a super(...) para inicializar atributos comunes.
- Sobrescribí algún método de Producto y llamá a super en su interior para reutilizar parte de la lógica.

1. Interfaces:

- Creá la interfaz Descontable con el método aplicarDescuento(double porcentaje).
- Hacé que Te y Cafe la implementen.
- Probá aplicar descuentos a diferentes productos.